

Aula 04

Proposições lógicas e tabela-verdade dos conectivos

Cinco conectivos, cinco regras — o vocabulário que transforma intuição em cálculo.

$\neg$ $\wedge$ $\vee$ $\rightarrow$ $\leftrightarrow$

Nesta aula



Os cinco conectivos

Negação, conjunção, disjunção, condicional e bicondicional.



Tabela-verdade de cada um

O valor lógico resultante para toda combinação possível.



Precedência entre conectivos

Qual conectivo é avaliado primeiro — igual em aritmética.



Como isso vira código

Os mesmos conectivos, agora em C# e Python.

Os cinco conectivos

Cada conectivo combina o valor lógico de uma ou duas proposições para produzir um novo valor lógico.



Negação

“não”



Conjunção

“e”



Disjunção

“ou”



Condicional

“se... então”



Bicondicional

“se e somente se”

Vamos ver a tabela-verdade de cada um, um por um.

Negação $\neg$

Inverte o valor lógico da proposição: “não p”.

p : “estudei” $\neg p$: “não estudei”

É o único conectivo unário — atua sobre uma única proposição, não duas.

p	$\neg p$
V	F
F	V

Conjunção $\wedge$ (e)

Só é verdadeira quando as duas proposições são verdadeiras.

$p \wedge q$: "estudei e dormi bem"

Basta uma parte falsa para a conjunção toda falhar — pense em pré-requisitos que precisam ser todos cumpridos.

p	q	$p \wedge q$
V	V	V
V	F	F
F	V	F
F	F	F

Disjunção $\vee$ (ou)

É verdadeira quando pelo menos uma das proposições é verdadeira.

$p \vee q$: "estudei ou dormi bem"

Só é falsa quando as duas partes são falsas — é a versão "inclusiva" do ou, aceita as duas ao mesmo tempo.

p	q	$p \vee q$
V	V	V
V	F	V
F	V	V
F	F	F

Condicional $\rightarrow$ (se... então)

p	q	$p \rightarrow q$
V	V	V
V	F	F
F	V	V
F	F	V



O mais contraintuitivo

“Se p então q” só é falsa quando p é verdadeira e a promessa não se cumpre (q falsa). Se p já é falsa, o condicional é sempre verdadeiro.

vacuamente verdadeiro

“Se eu fosse rico, compraria um avião” não é mentira só porque eu não sou rico — a promessa nunca foi testada.



“Se chove, a rua fica molhada”

p: “chove”

q: “a rua fica molhada”

fórmula: $p \rightarrow q$

V

V



V

Chove e a rua molha

A promessa se cumpriu.

V

F



F

Chove e a rua não molha

A promessa foi quebrada.

F

V



V

Não chove, mas a rua molha (ex.: lavaram a calçada)

A implicação só fala do que acontece quando chove.

F

F



V

Não chove e a rua não molha

Verdadeira, vacuamente.

Bicondicional $\leftrightarrow$

“se e somente se”

É verdadeira quando as duas proposições têm o mesmo valor lógico — ambas verdadeiras ou ambas falsas.

$p \leftrightarrow q$: “aprovado se e somente se nota ≥ 6 ”

É como uma via de mão dupla: $p \rightarrow q$ E $q \rightarrow p$, ao mesmo tempo.

p	q	$p \leftrightarrow q$
V	V	V
V	F	F
F	V	F
F	F	V



“Está frio se e somente se a temperatura está abaixo de 15°C”

p: “está frio” q: “temperatura < 15°C”

p	q	$p \leftrightarrow q$	Interpretação
V	V	V	Consistente: está frio e a temperatura confirma
V	F	F	Contradição: “está frio”, mas a temperatura diz que não
F	V	F	Contradição: a temperatura diz frio, mas você diz que não está
F	F	V	Consistente: nem está frio, nem a temperatura indica frio

JUNTANDO TUDO

As cinco tabelas, de uma vez

A tabela mestra: todas as combinações de p e q , e o resultado de cada conectivo.

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
V	V	F	V	V	V	V
V	F	F	F	V	F	F
F	V	V	F	V	V	F
F	F	V	F	F	V	V

Boa notícia: $\neg p$ só depende de p — por isso repete o mesmo valor nas duas linhas em que q muda.

Reconhecendo conectivos em português

Nem sempre a palavra do conectivo aparece de forma óbvia.

$\wedge$

(e)

“e”, “mas”, “porém”, “apesar de”, vírgula entre duas afirmações

$\vee$

(ou)

“ou”, “ou então”

$\rightarrow$

(se...então)

“se... então”, “quando”, “desde que”

$\leftrightarrow$

(se e somente se)

“se e somente se”, “é equivalente a”, “exatamente quando”

$\neg$

(não)

“não”, “nunca”, prefixos como “in-”, “im-”, “des-”

Traduzindo frases reais

“Estou cansado, mas vou estudar.”

o “mas” aqui funciona como “e”.

$$p \wedge q$$

“Desde que você tenha o crachá, pode entrar.”

“desde que” introduz uma condicional.

$$\text{crachá} \rightarrow \text{entrada}$$

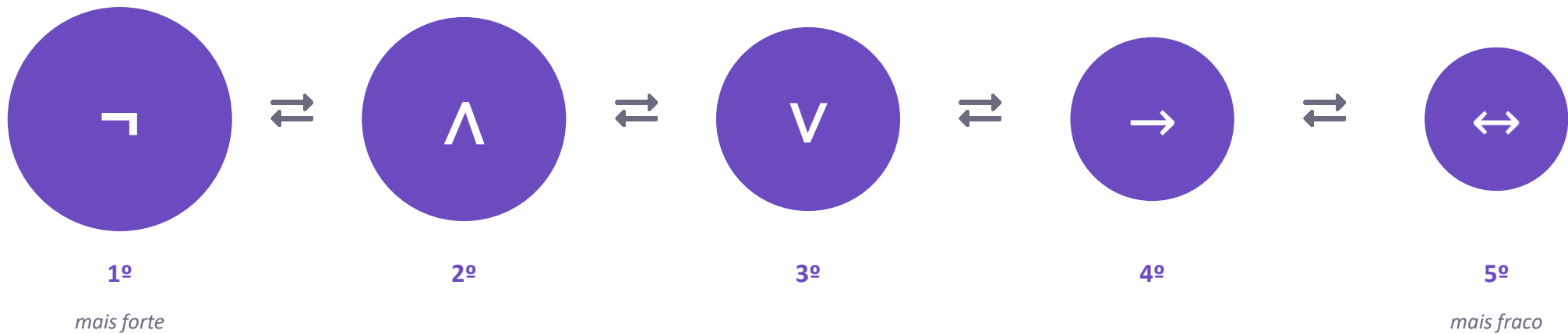
“Você aprova exatamente quando tira nota ≥ 6 .”

“exatamente quando” marca o bicondicional.

$$\text{aprovado} \leftrightarrow (\text{nota} \geq 6)$$

Precedência dos conectivos

Assim como $\times$ tem prioridade sobre $+$ na aritmética, os conectivos têm uma ordem de avaliação.



Na dúvida, use parênteses: eles deixam a fórmula legível para qualquer leitor — inclusive você mesmo, duas semanas depois.

Conectivos em código

Os símbolos que você acabou de ver, na sintaxe de cada linguagem.

SÍMBOLO



PYTHON



C#

 $\neg p$

not p

!p

 $p \wedge q$

p and q

p && q

 $p \vee q$

p or q

p || q

 $p \rightarrow q$

(not p) or q

(!p) || q

 $p \leftrightarrow q$

p == q

p == q

Python e C# não têm operador nativo para $\rightarrow$ e $\leftrightarrow$ — nós os construímos a partir dos operadores existentes.

PRÁTICA

Implementando $\rightarrow$ e $\leftrightarrow$

As duas funções que faltavam, testadas para as quatro combinações de p e q.

Conectivos.cs



```
static bool Implica(bool p, bool q) => !p || q;
static bool Bicondicional(bool p, bool q) => p == q;

foreach (var p in new[] { true, false })
foreach (var q in new[] { true, false })
{
    Console.WriteLine(
        $"{p} {q} {Implica(p,q)} {Bicondicional(p,q)}");
}
```

conectivos.py



```
def implica(p, q):
    return (not p) or q

def bicondicional(p, q):
    return p == q

for p in (True, False):
    for q in (True, False):
        print(p, q, implica(p, q), bicondicional(p, q))
```




Hora de praticar

Quatro exercícios: tabela-verdade, precedência, tradução e um desafio de código.



EXERCÍCIO 1

Tabela-verdade

Monte a tabela-verdade completa de $p \rightarrow q$. Compare com a tabela de $\neg p \vee q$. O que você percebe?

p	q	$p \rightarrow q$	$\neg p \vee q$
V	V	?	?
V	F	?	?
F	V	?	?
F	F	?	?



GABARITO · EXERCÍCIO 1

Tabela-verdade

p	q	$p \rightarrow q$	$\neg p \vee q$
V	V	V	V
V	F	F	F
F	V	V	V
F	F	V	V

Percebeu? As colunas $p \rightarrow q$ e $\neg p \vee q$ são idênticas — são logicamente equivalentes. É exatamente por isso que implementamos $\rightarrow$ em código como $(\text{not } p) \text{ or } q$.



EXERCÍCIO 2

Precedência

Sem usar parênteses adicionais, avalie a fórmula abaixo. Depois reescreva-a usando parênteses para deixar a ordem explícita.

$$\neg p \wedge q \vee r$$

VALORES

$$p = \text{V}$$

$$q = \text{F}$$

$$r = \text{V}$$

Resultado e fórmula com parênteses explícitos:



GABARITO · EXERCÍCIO 2

Precedência

1

$\neg p$

$\neg V = F$ (negação primeiro)

2

$F \wedge q$

$F \wedge F = F$ (conjunção antes da disjunção)

3

$F \vee r$

$F \vee V = V$ (disjunção por último)

Resultado: **V** Com parênteses explícitos: **$(\neg p \wedge q) \vee r$**



EXERCÍCIO 3

Traduzindo

“Se estudo e durmo bem, então passo na prova.”

Identifique p, q, r e o(s) conectivo(s) usado(s).

p =

q =

r =

Fórmula com conectivos:



GABARITO · EXERCÍCIO 3

Traduzindo

p

“estudo”

q

“durmo bem”

r

“passo na prova”

$$(p \wedge q) \rightarrow r$$

Conectivos usados: $\wedge$ (conjunção) e $\rightarrow$ (condicional).



EXERCÍCIO 4 · DESAFIO

Bicondicional, na mão

Implemente `bicondicional_v2(p, q)` usando apenas `and`, `or` e `not` (sem usar `==`). Teste contra o bicondicional da aula para as 4 combinações de `p` e `q`.

dica: $p \leftrightarrow q$ é verdadeira quando $(p \text{ e } q)$ ou $(\text{nem } p, \text{ nem } q)$

```
● ● ● bicondicional_v2.py
```

```
def bicondicional_v2(p, q):  
    return _____
```

Teste: escreva as 4 combinações de `p` e `q` e confira se o resultado é igual ao `bicondicional(p, q)`.



GABARITO · EXERCÍCIO 4

Bicondicional, na mão

 bicondicional_v2.py



```
def bicondicional_v2(p, q):  
    return (p and q) or (not p and not q)
```

Equivalente em C#: `static bool BicondicionalV2(bool p, bool q) => (p && q) || (!p && !q);`

p	q	bicondicional	bicondicional_v2
V	V	V	V
V	F	F	F
F	V	F	F
F	F	V	V

Onde estamos no semestre



Depois de dominar os cinco conectivos, é hora de combiná-los em exercícios mais longos.

Referências

Livros — teoria



ROSEN, K. H.

Matemática Discreta e suas Aplicações. 7. ed.
AMGH/McGraw-Hill — cap. Lógica Proposicional
(conectivos e tabelas-verdade).



GERSTING, J. L.

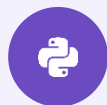
Fundamentos Matemáticos para a Ciência da
Computação. LTC — cap. Lógica.



LIPSCHUTZ, S.; LIPSON, M.

Matemática Discreta (Coleção Schaum).
Bookman — cap. Lógica Proposicional.

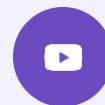
Documentação e vídeos



Python — operações booleanas

[Operações booleanas \(and/or/not\)](https://docs.python.org/3/library/stdtypes.html#boolean-operations) — docs.python.org

[Operadores de comparação \(==\)](https://docs.python.org/3/library/stdtypes.html#comparisons) — docs.python.org



Vídeos

[Conectivos lógicos e tabelas-verdade](#) — explicação passo a passo.

[Canal Prof. Ferretto](#) — lógica matemática com muitos exercícios resolvidos.



PRÓXIMA PARADA

Aplicação das tabelas-verdade em exercícios

Vamos praticar combinando conectivos em exercícios mais longos.

Guarde o slide das “cinco tabelas juntas” — ele vai ser sua cola pessoal.